Laboratorio 5 — Modelado, simulación y validación de Statecharts con ITEMIS

Edición utilizada: ITEMIS CREATE 5.6.3

Los valores «esperado» se registraron antes de simular. En todos los casos, lo observado en la simulación coincidió con lo esperado.

HITO 1 — PRIMER MODELO EJECUTABLE

PARTE A — UN LED

Paso	Estímulo	Estado esperado	Salida esperada	Estado y salida observados	¿Cumple?
0	Inicializar	LED_APAGADO	led_encendido = false	LED_APAGADO; led_encendido = false	Sí
1	pulsacion	LED_ENCENDIDO	led_encendido = true	LED_ENCENDIDO; led_encendido = true	Sí
2	pulsacion	LED_APAGADO	led_encendido = false	LED_APAGADO; led_encendido = false	Sí
3	Sin evento	LED_APAGADO	led_encendido = false	LED_APAGADO; led_encendido = false	Sí
4	pulsacion	LED_ENCENDIDO	led_encendido = true	LED_ENCENDIDO; led_encendido = true	Sí

En el paso 3 no hay evento: sin `pulsacion` no se dispara ninguna transición, aunque pase el tiempo.

PARTE B — DOS LED INDEPENDIENTES

Paso	Evento	LED 1 esperado	LED 2 esperado	Estados activos y salidas observados
0	Inicializar	Apagado	Apagado	LED1_APAGADO + LED2_APAGADO; led_1_encendido = false, led_2_encendido = false
1	pulsacion_1	Encendido	Apagado	LED1_ENCENDIDO + LED2_APAGADO; led_1_encendido = true, led_2_encendido = false
2	pulsacion_2	Encendido	Encendido	LED1_ENCENDIDO + LED2_ENCENDIDO; led_1_encendido = true, led_2_encendido = true
3	pulsacion_1	Apagado	Encendido	LED1_APAGADO + LED2_ENCENDIDO; led_1_encendido = false, led_2_encendido = true
4	pulsacion_2	Apagado	Apagado	LED1_APAGADO + LED2_APAGADO; led_1_encendido = false, led_2_encendido = false

Estados activos esperados (uno por región): LED1_APAGADO/LED1_ENCENDIDO en la región 1 y LED2_APAGADO/LED2_ENCENDIDO en la región 2.

FSM plana vs. regiones ortogonales. Una FSM plana necesita un estado por cada combinación de los dos LED: 2 × 2 = 4 estados, con transiciones entre todos ellos. Con regiones ortogonales se dibujan 2 + 2 = 4 estados, pero la diferencia crece con la escala: para *n* LED la FSM plana necesita 2^n estados y el modelo ortogonal solo 2n.

Dos estados activos simultáneos. El sistema es la composición de dos máquinas concurrentes. Cada región mantiene su propio estado activo y la configuración global es la combinación de ambos. Son independientes porque `pulsacion_1` solo la consume la región 1 y `pulsacion_2` solo la región 2.

HITO 3 — PRUEBAS FUNCIONALES Y SEGURIDAD

Las pruebas se ejecutaron reinicializando el modelo antes de cada caso. Las salidas de operación son `microondas_activo` y `grill_activo`; los indicadores son `indicador_microondas` e `indicador_grill`.

Caso	Secuencia	Estados y salidas esperados	Observado	¿Cumple?
P1	Sin eventos	Detenido, Cerrada, Microondas; microondas_activo = false, grill_activo = false; indicador_microondas = true, indicador_grill = false	Coincide con lo esperado	Sí
P2	startStop → startStop	Tras el 1.º: Funcionando, microondas_activo = true. Tras el 2.º: detenido, ambas salidas false	Coincide con lo esperado en ambos pasos	Sí
P3	abrir_puerta → startStop	Tras abrir: Detenido, Abierta, salidas false. Tras startStop: sin cambios (no inicia)	Coincide con lo esperado; no inició	Sí
P4	startStop → abrir_puerta	Tras startStop: Funcionando, microondas_activo = true. Tras abrir: Detenido, Abierta, ambas salidas false	Coincide con lo esperado en ambos pasos	Sí
P5	modo → startStop → abrir_puerta	Tras modo: Grill, indicador_grill = true. Tras startStop: Funcionando, grill_activo = true. Tras abrir: Detenido, Abierta, ambas salidas false, Grill sigue seleccionado	Coincide con lo esperado en los tres pasos	Sí
P6	abrir_puerta → modo → cerrar_puerta	Tras abrir: Abierta. Tras modo: Grill seleccionado, sin operar. Tras cerrar: Cerrada, Detenido, no inicia	Coincide con lo esperado; no inició	Sí
P7	startStop → abrir_puerta → cerrar_puerta	Tras startStop: Funcionando. Tras abrir: Detenido, Abierta, salidas false. Tras cerrar: Detenido, Cerrada, salidas false	Coincide con lo esperado en los tres pasos	Sí
P8	startStop → modo	Tras startStop: Funcionando, microondas_activo = true. Tras modo: sin cambios, sigue funcionando en Microondas	Coincide con lo esperado; el modo no cambió	Sí

Nota: con el modelo final (Hito 4), después de la detención voluntaria del caso P2 el sistema queda en Detenido con `aviso_activo = true` durante 2 s y luego `aviso_activo` vuelve a false. Las salidas de operación son false en ambos momentos.

CONDICIONES DE SEGURIDAD

1. `microondas_activo` y `grill_activo` nunca son `true` simultáneamente.
2. Con la puerta abierta, ambas salidas permanecen `false`.

Decisiones del modelo que ayudan a garantizarlo:

- **Modo exclusivo.** El modo se representa con dos estados mutuamente excluyentes en la región `Modo` (Microondas o Grill). Al empezar a funcionar, cada salida se asigna según el estado activo de esa región, por lo que no pueden valer `true` a la vez.
- **Guarda de arranque.** La transición de inicio es `startStop [active(Puerta.Cerrada)]`: con la puerta abierta, `startStop` se ignora.
- **Interrupción inmediata.** Existe una transición `abrir_puerta` desde Funcionando que vuelve a Detenido. El mismo evento lo consumen la región `Puerta` (pasa a Abierta) y la región `Operacion` (detiene), en el mismo paso.
- **Apagado de salidas.** El estado `Detenido` tiene `entry / microondas_activo = false; grill_activo = false`, así que las salidas de operación se apagan siempre que se entra a Detenido, ya sea por `startStop` o por `abrir_puerta`.
- **Cambio de modo protegido.** `modo [active(Operacion.Detenido)]` impide cambiar el modo mientras funciona.
- **Eventos redundantes.** `abrir_puerta` con la puerta abierta y `cerrar_puerta` con la puerta cerrada no tienen transición, así que no producen efectos.

Las pruebas finitas no constituyen una demostración exhaustiva.

DIAGNÓSTICO DE UN ERROR

Se guardó una copia funcional y se introdujo deliberadamente un error de comportamiento que no impide simular.

Elemento	Detalle
Modificación	En la transición Detenido → Funcionando, se quitó la guarda: antes `startStop [active(Puerta.Cerrada)]`, después `startStop`
Requisito que dejó de cumplirse	Con la puerta abierta, ambas salidas deben permanecer false (condición de seguridad 2) y `startStop` no debe iniciar con la puerta abierta
Caso que lo detectó	P3 (abrir_puerta → startStop): se esperaba «no inicia»; se observó «Funcionando con la puerta abierta» y una salida de operación en true
Por qué no lo detectó el editor	El modelo es sintácticamente correcto: Problems no informa errores, pero el comportamiento es incorrecto
Corrección	Se restauró la guarda `[active(Puerta.Cerrada)]`
Reverificación	P3 volvió a dar «Detenido, Abierta, salidas false»

HITO 4 — COMPORTAMIENTO TEMPORIZADO

No se creó un estado exclusivo para el aviso: el aviso es un período dentro de `Detenido`, controlado por la variable booleana `aviso_activo` y una reacción temporizada local.

Implementación:

- La transición `Funcionando → Detenido` por detención voluntaria es `startStop / aviso_activo = true`.
- La transición `Funcionando → Detenido` por puerta es `abrir_puerta` y no toca `aviso_activo`, por lo que no genera aviso.
- En `Detenido`:
  - `entry / microondas_activo = false; grill_activo = false`
  - `after 2 s [aviso_activo] / aviso_activo = false`
  - `abrir_puerta / aviso_activo = false`
  - `exit / aviso_activo = false`

Prueba	Secuencia	Esperado	Observado (aviso_activo, salidas, estados)	¿Cumple?
T1	startStop → startStop → esperar 2 s	Tras la detención: Detenido, aviso_activo = true, salidas false. A los 2 s: aviso_activo = false, sigue en Detenido con el mismo modo	Coincide con lo esperado; el aviso se apagó solo a los 2 s	Sí
T2	startStop → abrir_puerta	Detenido, Abierta, salidas false; aviso_activo nunca pasa a true	Coincide con lo esperado; no hubo aviso	Sí
T3	startStop → startStop → abrir_puerta (antes de 2 s)	aviso_activo = false de inmediato; Detenido, Abierta	Coincide con lo esperado; el aviso se canceló de inmediato	Sí
T4	startStop → startStop → startStop (antes de 2 s)	aviso_activo = false (por el exit de Detenido); Funcionando con la salida de operación en true	Coincide con lo esperado; el aviso se canceló y comenzó otra operación	Sí
T5	startStop → startStop → modo (a ~0,8 s)	El modo cambia; aviso_activo sigue en true y se apaga ~1,2 s después del cambio (2 s desde la detención original)	Coincide con lo esperado; el modo cambió y el aviso terminó 2 s después de la detención original	Sí

Qué ocurre con la referencia temporal al abandonar anticipadamente el comportamiento temporizado. La reacción `after 2 s` está asociada a la permanencia en `Detenido`: el conteo empieza al entrar y se cancela al salir. Si se inicia de nuevo antes de los 2 s, la transición `startStop` abandona `Detenido`, el temporizador se cancela y el `exit` apaga `aviso_activo`, de modo que el aviso no puede terminar después por error. Al volver a Detenido, el conteo arranca de cero. Abrir la puerta durante el aviso lo cancela con la reacción local `abrir_puerta / aviso_activo = false`, que no sale del estado y por lo tanto no reinicia el conteo.

Cómo se evitó que cambiar de modo reiniciara el plazo. El cambio de modo ocurre en la región `Modo`, independiente de `Operacion`. Esa transición no sale ni reingresa en `Detenido`, así que su temporizador no se modifica y sigue contando desde la detención original. Además, como durante el aviso el sistema sigue en `Detenido`, la guarda `modo [active(Operacion.Detenido)]` ya permite el cambio sin necesidad de modificarla.

HITO 5 — PREPARACIÓN DE LA INTERFAZ PARA STM32

Arquitectura de referencia: **entradas físicas → código C de adaptación → Statechart → código C de adaptación → salidas físicas.** El modelo permanece independiente de registros, pines y llamadas HAL.

Elemento	Información
Eventos de entrada	`startStop`: solicitud de inicio/detención; pulsador. `modo`: solicitud de alternar Microondas/Grill; pulsador. `abrir_puerta`: la puerta pasó a abierta; sensor o final de carrera (flanco). `cerrar_puerta`: la puerta pasó a cerrada; mismo sensor (flanco opuesto)
Variables de entrada	No hay. Todo ingresa como eventos instantáneos; el estado de la puerta lo conserva el modelo en la región `Puerta`
Salidas del modelo	`microondas_activo`: operación de microondas; LED o relé. `grill_activo`: operación de grill; LED o relé. `indicador_microondas`: modo Microondas seleccionado; LED. `indicador_grill`: modo Grill seleccionado; LED. `aviso_activo`: aviso de detención de 2 s; LED o buzzer
Servicio temporal	El aviso de 2 s necesita una base de tiempo periódica. Puede proveerla el SysTick (por ejemplo `HAL_GetTick`) o un temporizador (TIM) que genere un tick regular, usado para avanzar el reloj del Statechart
Ejecución del modelo	Se ejecuta periódicamente (por ejemplo cada 10 ms) y cada vez que el código de adaptación genera un evento. En cada ejecución procesa los eventos pendientes y avanza el tiempo para las reacciones `after`
Código C de adaptación	Lee los pulsadores y el sensor de puerta, aplica antirrebote y detección de flancos, y convierte eso en eventos `startStop`, `modo`, `abrir_puerta` y `cerrar_puerta`. Provee la referencia de tiempo. Después de ejecutar el modelo, lee las salidas booleanas y las escribe en los GPIO (LED, relé, buzzer)

Separación de responsabilidades. La *lógica de control* (qué estado corresponde y qué salidas se activan) vive en el Statechart. La *adquisición de entradas* (leer pines, filtrar rebotes, detectar flancos) la hace el código C de adaptación, que entrega al modelo solo eventos con significado. La *actuación física* (escribir GPIO) también está fuera del modelo, que solo publica valores booleanos. Así el modelo se puede simular y probar en ITEMIS sin hardware, y cambiar el hardware no obliga a modificar la lógica.

PREGUNTAS DE ANÁLISIS

1. ¿En qué se diferencia que ITEMIS no informe errores de que el modelo cumpla los requisitos?
La validación del editor solo verifica sintaxis y coherencia (referencias, tipos, estados iniciales). No sabe qué debe hacer el horno. Un modelo sin errores puede comportarse mal, como en el error del Hito 3, donde quitar una guarda dejó el modelo válido pero inseguro. Que cumpla los requisitos solo se comprueba con pruebas contra el comportamiento esperado.

2. Organización de estados y regiones.
El horno es un estado ortogonal `Hornomicroondas` con tres regiones: `Operacion` (Detenido y Funcionando; el aviso se resuelve con la variable `aviso_activo` dentro de Detenido), `Puerta` (Cerrada, Abierta) y `Modo` (Microondas, Grill). Se usaron regiones ortogonales porque las tres cosas son independientes entre sí: la puerta puede estar abierta o cerrada con cualquier modo y en cualquier estado de operación. Una FSM plana necesitaría hasta 2 × 2 × 2 = 8 combinaciones de estados (algunas inalcanzables, como Funcionando con la puerta abierta) con transiciones duplicadas, contra 2 + 2 + 2 = 6 estados en el modelo ortogonal. Las regiones se coordinan con guardas `active(...)`.

3. ¿Qué significa que dos regiones ortogonales estén activas simultáneamente? ¿Por qué no implica ejecución paralela ni dos tareas de FreeRTOS?
Significa que el sistema tiene un estado activo en cada región a la vez; la configuración global es la combinación. Es un concepto del *modelo*, no de la ejecución: el código generado procesa las regiones de forma secuencial dentro de un mismo ciclo, en un orden determinado, y dentro de una única llamada. No hacen falta dos hilos ni dos tareas de FreeRTOS.

4. ¿Qué características del modelo impiden activar simultáneamente microondas y grill, o funcionar con puerta abierta?
El modo es exclusivo (dos estados de una misma región); las salidas se asignan según el modo activo al entrar a Funcionando y se apagan al entrar a Detenido; el arranque tiene la guarda `active(Puerta.Cerrada)`; `abrir_puerta` detiene la operación de inmediato; y cerrar la puerta no reanuda el funcionamiento.

5. ¿En qué se diferencia un evento instantáneo de una condición persistente?
Un evento (`startStop`, `abrir_puerta`) ocurre en un instante, se consume al procesarse y no queda almacenado. Una condición persistente (puerta abierta, modo seleccionado, `aviso_activo`) se mantiene en el tiempo y se representa con estados o variables. Por eso la puerta se modela como estado (Abierta/Cerrada) y las guardas consultan ese estado, en lugar de depender de un evento que ya pasó.

6. ¿Cómo se ejecutan las reacciones temporizadas en la simulación y qué necesitará proporcionar la aplicación STM32?
En la simulación, ITEMIS avanza un reloj virtual, y `after 2 s` se dispara cuando el tiempo transcurrido en el estado alcanza 2 s, sin esperas bloqueantes. En el STM32, la aplicación deberá proporcionar la base de tiempo (SysTick o un timer) y llamar periódicamente al modelo para que avance su reloj y evalúe las reacciones temporizadas.

7. Si la señalización se implementara con otro Statechart o un módulo C independiente, ¿qué información debería intercambiar con el control principal?
Del control principal hacia la señalización: el estado a mostrar (modo seleccionado, funcionando, aviso activo, puerta abierta) o directamente las salidas booleanas (`indicador_microondas`, `indicador_grill`, `microondas_activo`, `grill_activo`, `aviso_activo`). En sentido contrario no debería ser necesario nada, para que la señalización no pueda alterar el control. Si necesitara enviar algo, sería mediante eventos con significado propio.

8. ¿Qué conviene resolver con estados, variables o acciones del modelo y qué mediante código C externo?
En el modelo: la lógica de control (modos, puerta, funcionamiento, aviso temporizado y coherencia de las salidas). En código C externo: lo que depende del hardware, como leer pines, antirrebote, detección de flancos, base de tiempo, escritura de GPIO y manejo de periféricos. Regla práctica: si es una decisión sobre el comportamiento del horno, va al modelo; si es una operación sobre un pin o un periférico, va al código de adaptación.